

CREACIÓN DEL PROYECTO DE BACKEND



■ Guía paso a paso: Creación del backend con Java empresarial (Jakarta EE)

✦ Introducción

En esta guía aprenderás a construir desde cero un backend profesional utilizando Jakarta EE como plataforma empresarial. Este backend se integrará posteriormente con un cliente desarrollado en Angular, lo que te permitirá manejar información de personas de forma estructurada a través de servicios REST. Trabajaremos con Maven, configuraremos las dependencias necesarias y estableceremos la estructura base del proyecto incluyendo la capa de persistencia y una clase de entidad.

1. Crear el proyecto backend con NetBeans

Descripción:

Creamos un nuevo proyecto Maven en NetBeans configurado para usar Jakarta EE (Java empresarial).

Archivo: Proyecto Maven

Ruta:

C:\Cursos\AngularJakartaEE\personas-backend-java

Pasos:

- Abrimos Apache NetBeans
 - Vamos a **File > New Project**
 - Seleccionamos **Maven > Java Web Application**
 - Configuramos:
 - **Project Name:** personas-backend-java
 - **Location:** C:\cursos\AngularJakartaEE
 - **GroupId:** gm
 - **Version:** 1.0
 - Damos clic en **Finish**
 - Renombramos el proyecto a `personas-backend-java` si es necesario
-

2. Eliminar clases generadas automáticamente

Descripción:

Eliminamos las clases predeterminadas generadas por el arquetipo Maven para dejar limpio el proyecto.

Pasos:

- Dentro de `src/main/java`, eliminamos los paquetes generados automáticamente
 - Esto dejará el proyecto sin clases predefinidas
-

3. Configurar propiedades del proyecto

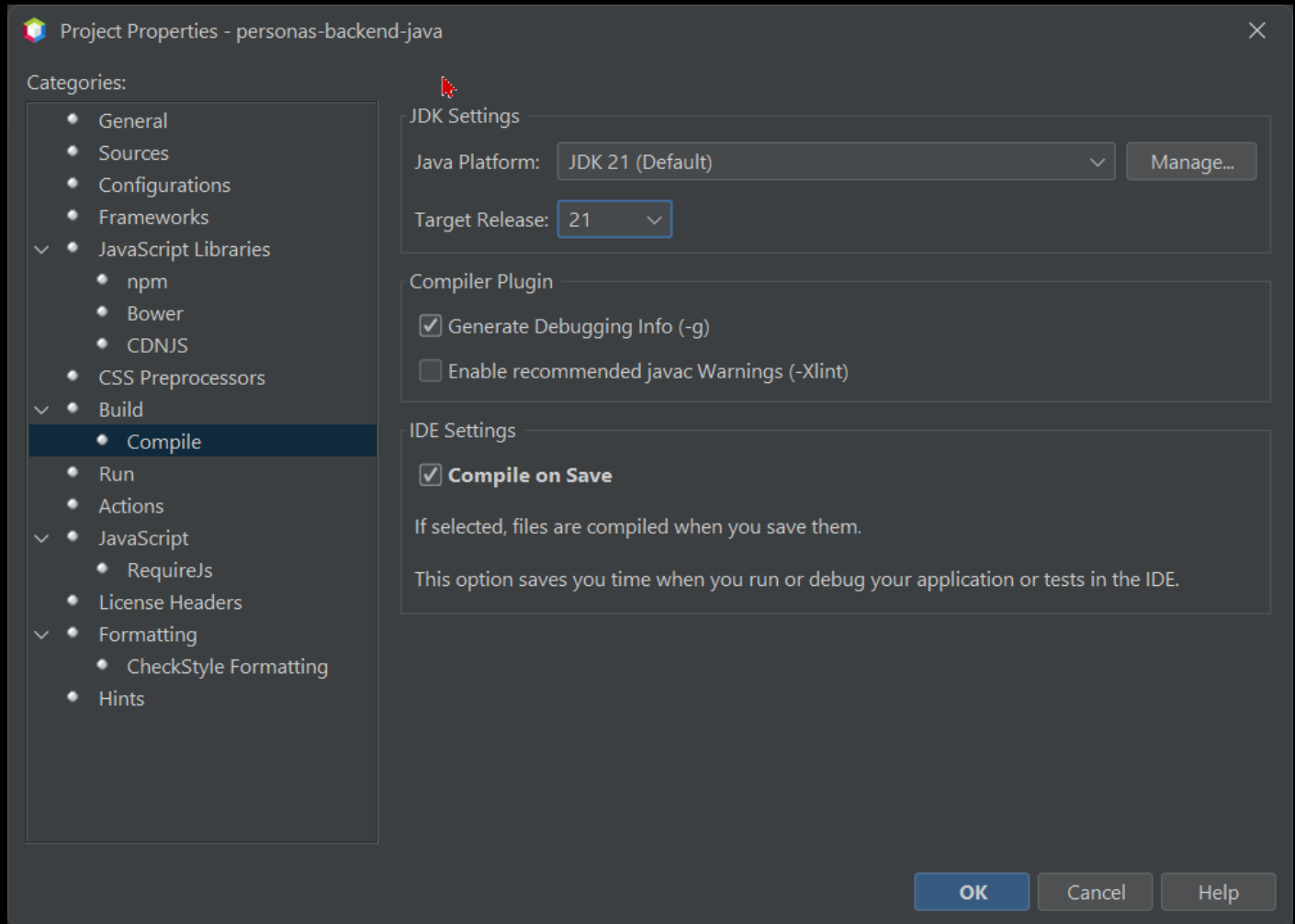
Descripción:

Damos clic derecho sobre el proyecto, seleccionamos propiedades:

Entramos a la opción de build > Compile

Y en esta pantalla seleccionamos el target release a la versión JDK 21.

Además, seleccionamos la opción de Compile on Save aceptando cualquier ventana extra que nos muestre para aplicar los cambios. Damos clic en ok.



4. Agregar dependencias necesarias

📁 Descripción:

Agregamos las bibliotecas necesarias para trabajar con servicios REST (JAX-RS).

📄 Archivo:

personas-backend-java/pom.xml

✂ Fragmento de código:

```
<dependency>
  <groupId>org.glassfish.jersey.core</groupId>
  <artifactId>jersey-client</artifactId>
  <version>3.1.10</version>
</dependency>
```

✂ Se utiliza Jersey como implementación de JAX-RS.

5. Configurar archivo de persistencia

✿ Descripción:

Definimos el archivo `persistence.xml` para conectar con la base de datos usando JTA y un `DataSource` configurado previamente en GlassFish.

📄 Archivo:

personas-backend-java/src/main/resources/META-INF/persistence.xml

✂ Fragmento de código:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.0" xmlns="https://jakarta.ee/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">
  <!-- Define Persistence Unit -->
  <persistence-unit name="PersonaPU">
    <jta-data-source>jdbc/PersonaDb</jta-data-source>
  </persistence-unit>
</persistence>
```

✂ Usa el nombre exacto del `DataSource` configurado previamente en GlassFish: `jdbc/PersonaDB`. (Debes respetar mayúsculas y minúsculas)

6. Modificar archivo `index.html`

Archivo:

`personas-backend-java/src/main/webapp/index.html`

✚ Modificamos el título del archivo html, pero sólo es como referencia, no se va a mostrar de momento.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Aplicacion Personas Rest Web Services</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  </head>
  <body>
    <h1>Aplicacion Personas Rest Web Services</h1>
  </body>
</html>
```

7. Crear clase de entidad `Persona`

Descripción:

Creamos una clase de entidad que represente la tabla `persona` en la base de datos.

Archivo:

`personas-backend-java/src/main/java/gm/domain/Persona.java`

Código:

```
package gm.domain;

import jakarta.persistence.*;
import java.io.Serializable;

@Entity
@NamedQueries({
    @NamedQuery(name = "Persona.encontrarTodasPersonas", query = "SELECT p FROM Persona p ORDER BY p.idPersona")
})
public class Persona implements Serializable{
```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "id_persona")
private int idPersona;

private String nombre;

public Persona(){

}

public Persona(int idPersona){
    this.idPersona = idPersona;
}

public Persona(int idPersona, String nombre){
    this.idPersona = idPersona;
    this.nombre = nombre;
}

public int getIdPersona() {
    return idPersona;
}

public void setIdPersona(int idPersona) {
    this.idPersona = idPersona;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

@Override
public String toString() {
    return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + '}';
}

}
```

- ✂ Se usa la anotación `@Entity` para indicar que es una entidad JPA.
 - ✂ El atributo `id_persona` está anotado como llave primaria y con `GenerationType.IDENTITY`.
-

Conclusión

En esta lección construimos desde cero la base de una aplicación backend con Java empresarial. Aprendiste a crear el proyecto con Maven, configurar sus propiedades, definir dependencias, conectar con la base de datos usando JTA y crear una clase de entidad mapeada correctamente. Este backend estará listo para exponer servicios REST en futuras lecciones e integrarse con un frontend en Angular.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)